

AI Principles and Practice · Lecture 1

Data Analysis and the ML Workflow

Week 1 · Phase I — Foundations and Ethics · LOs: LO2, LO5

Lecturer: Indrakshi

1. **The end-to-end ML workflow**
2. **Data types** and encoding
3. **Summary statistics** — centre, spread, shape
4. **Distributions & visualisation**
5. **Correlation** — and its traps
6. **Data processing** — scaling, missingness, outliers, augmentation
7. **Leakage-safe pipelines**
8. **Governance & documentation**

Every concept comes with
a **mathematical definition**, a short **Python snippet**, **how to run it**, and the **output** — and a figure where a picture beats a paragraph.

What you'll be able to do

Module outcomes today

- **LO2** — distinguish ML paradigms and place each in the workflow
- **LO5** — data governance, documentation, the ethics of data

Concrete skills

- Classify variables and read their distributions
- Quantify centre, spread, shape and association
- Detect missingness and outliers; augment scarce data
- Build a **leakage-safe** preprocessing pipeline

Your materials for this week

Slides

What we work through together in the 90-minute lecture.

Booklet

A book-length reference: every concept, derivation, full code, outputs.

Code

`make_assets.py` — one runnable script that reproduces every number and figure.

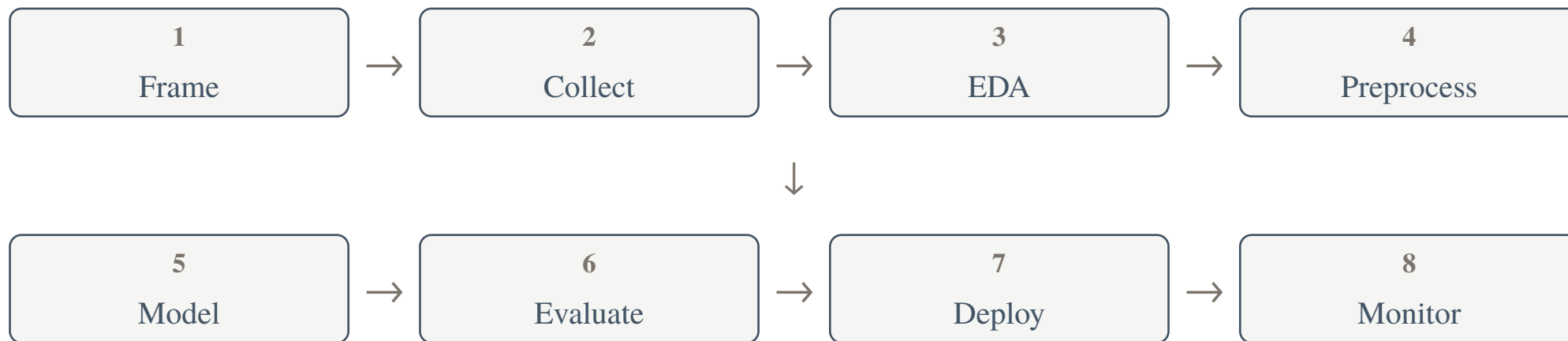
Reproducibility from day one

Every figure and output in these slides was produced by the script — fixed seed, no hand-tuning. You can re-run it and get the same results.

Section 1

The end-to-end ML workflow

The workflow is a loop, not a line



Monitoring and evaluation feed straight back into reframing and better data. **Weeks 1–5 live almost entirely in stages 1–4.**

Where the course maps onto the workflow

Workflow stage	Covered in
Data, EDA, preprocessing	Week 1 — today
Modelling — supervised learning	Week 2
Ethics, fairness, safety, governance	Weeks 3–4
Representations / unsupervised	Week 5
Neural networks & the mathematics	Weeks 7–8
Modern methods — transfer, transformers, diffusion	Weeks 9–11

Setup: a small, reproducible environment

setup.py

python

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer, KNNImputer
from sklearn.pipeline import Pipeline
```

```
$ python -V
```

console

```
Python 3.12 · numpy 2.4 · pandas 3.0 · scikit-learn 1.8
```

Our running example: a synthetic customers table

data.py

python

```
rng = np.random.default_rng(42)
n = 600
df = pd.DataFrame({
    "age":      rng.normal(40,12,n).round(),
    "income":
rng.lognormal(10.6,.45,n).round(),
    "tenure":  rng.integers(0,72,n),
    "plan":
rng.choice(["Basic","Standard",
            "Premium"], n),
    "satisfaction": rng.choice(
        ["Low","Medium","High"], n),
})
df.loc[rng.random(n)<.08,"income"] =
np.nan
```

df.head()

	age	income	tenure	plan	sat
0	44.0	50612.0	29	Standard	Medium
1	28.0	30949.0	14	Standard	Medium
2	49.0	71218.0	7	Premium	Medium
3	51.0	NaN	30	Basic	High
4	18.0	NaN	37	Premium	High

Synthetic, seeded, with deliberate gaps and outliers — the full builder is in the booklet.

First contact: profile before you model

profile.py

python

```
df[["age", "income", "monthly_charge"]].describe().round(2)
```

```
$ python profile.py
```

```
describe()
```

	age	income	monthly_charge	
count	600.00	547.00	570.00	
mean	39.82	53565.44	37.39	
std	11.42	112607.49	8.36	<- huge: outliers + skew
min	18.00	7771.00	21.25	
50%	40.00	40388.00	36.20	<- median << mean (income)
max	75.00	1342344.00	84.85	

Section 2

Data types and encoding

Know your variable types

Type	What it is	Example
Binary	Exactly two categories	churn: 0 / 1
Categorical (nominal)	Unordered groups	region, plan
Ordinal	Ordered, unequal gaps	satisfaction
Discrete	Countable integers	tenure (months)
Continuous	Any value in a range	age, income

Type decides the summary statistic, the plot, the **encoding**, and the model. Models eat numbers — categories must be encoded first.

Ordinal encoding — order carries meaning

Low $\rightarrow$ 0, Medium $\rightarrow$ 1, High $\rightarrow$ 2

a monotone map that **preserves order** — only valid when an order truly exists

encode_ordinal.py

python

```
from sklearn.preprocessing import  
OrdinalEncoder  
oe = OrdinalEncoder(  
    categories=[["Low", "Medium", "High"]])  
oe.fit_transform(df[["satisfaction"]][:4])
```

out

satisfaction	encoded
Medium	1
Medium	1
Medium	1
High	2

One-hot encoding — no false ordering

$\text{plan} \rightarrow [\text{Basic}, \text{Premium}, \text{Standard}] \in \{0, 1\}^3$

one column per category; exactly one is “hot” — use for **nominal** variables

encode_onehot.py

python

```
from sklearn.preprocessing import  
OneHotEncoder  
oh = OneHotEncoder(sparse_output=False,  
dtype=int)  
oh.fit_transform(df[["plan"]])[:4]
```

out

Basic	Premium	Standard
0	0	1
0	0	1
0	1	0
1	0	0

Section 3

Summary statistics

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad \text{median} = \text{middle value of sorted } x$$

the **mean** chases outliers; the **median** does not

centre.py

python

```
inc = df["income"].dropna()
inc.mean(), inc.median()
```

out

```
mean    = 53,565.4
median  = 40,388.0
```

The mean sits **32% above** the median — a textbook signature of right skew and outliers.

Spread: variance, standard deviation, IQR

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \quad s = \sqrt{s^2}, \quad \text{IQR} = Q_3 - Q_1$$

SD is in the data's units; IQR is the robust, outlier-resistant alternative

spread.py

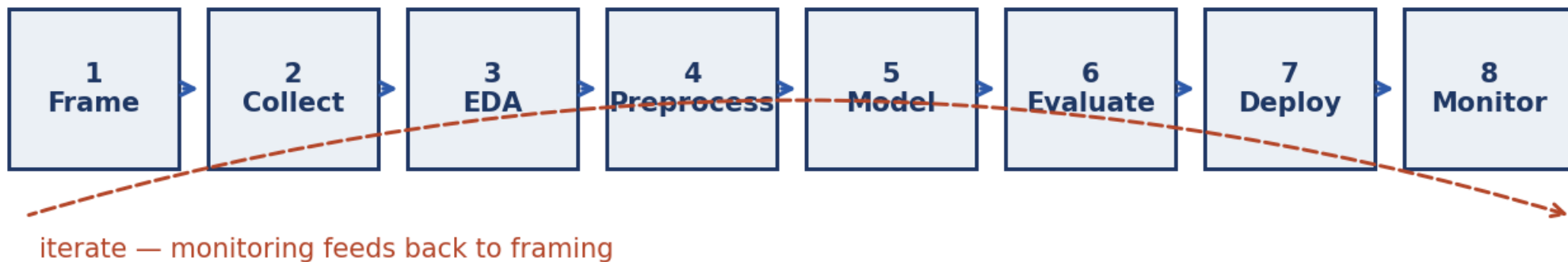
python

```
inc.std(ddof=1)
q1, q3 = inc.quantile([.25, .75])
iqr = q3 - q1
```

out

```
std = 112,607.5
IQR = 23,204.5
(Q1=30,143, Q3=53,348)
```

Why robustness matters



The mean (red) is dragged right by the tail; the median (navy dashed) stays with the bulk of customers.

Shape: skewness and kurtosis

$$g_1 = \frac{\frac{1}{n} \sum (x_i - \bar{x})^3}{s^3}, \quad g_2 = \frac{\frac{1}{n} \sum (x_i - \bar{x})^4}{s^4} - 3$$

g_1 = asymmetry (skew); g_2 = tailedness (excess kurtosis)

shape.py

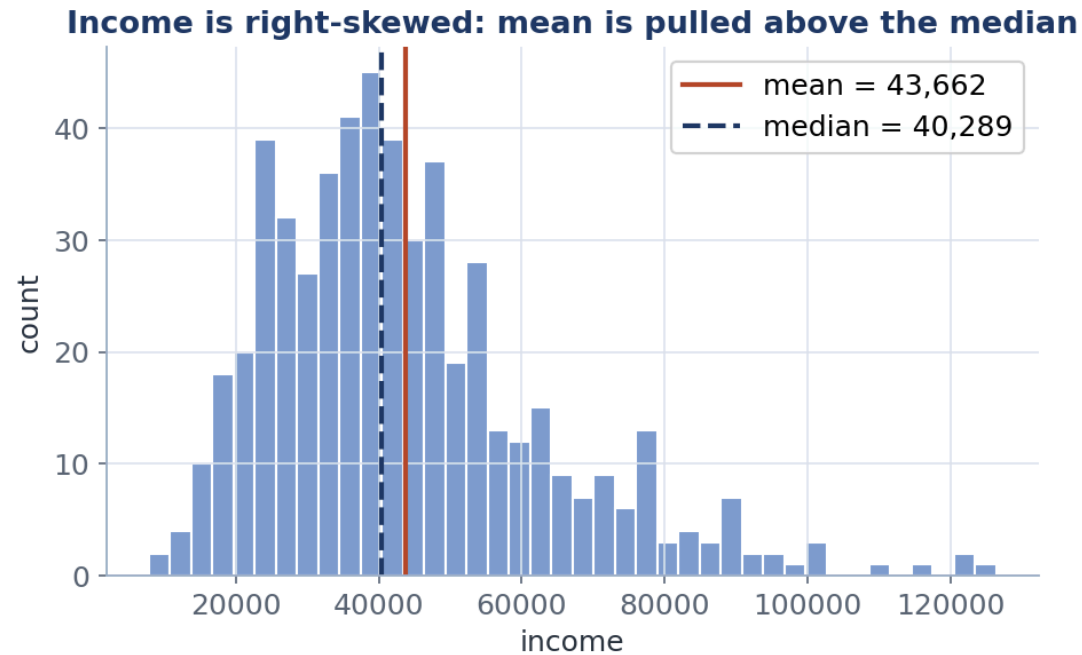
python

```
from scipy import stats
stats.skew(inc), stats.kurtosis(inc)
```

out

```
skew = 10.99    very right-skewed
kurt = 122.93   extremely heavy tails
```

The box plot, decoded



Box = IQR ($Q1-Q3$), line = median, whiskers reach $1.5 \cdot IQR$, points beyond are flagged.

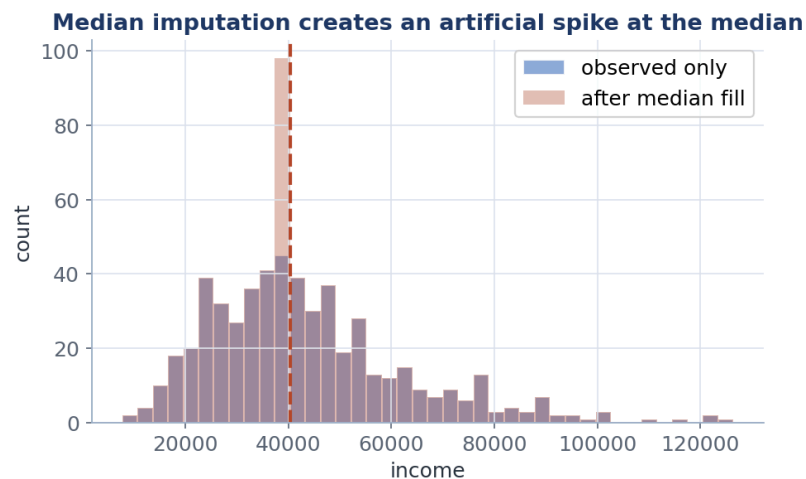
Section 4

Distributions and visualisation

Summary numbers are not enough

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

the Normal is fully described by μ, σ — but most real data is not Normal



Identical mean and standard deviation, yet one is unimodal and one is bimodal. Always plot.

Visualisation is a discipline, not decoration

Question	Reach for
Distribution of one variable	Histogram, KDE, box / violin plot
Relationship between two	Scatter plot, 2-D density
Comparison across groups	Grouped bar, box-by-category
Trend over time	Line chart
Many variables at once	Correlation heatmap, pair plot

Choose the chart for the **question**; label honestly; never let a default colour map decide the story.

Section 5

Correlation — and its traps

Pearson correlation: linear association

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2} \cdot \sqrt{\sum_i (y_i - \bar{y})^2}}$$

$r \in [-1, 1]$ — measures **linear** association only

pearson.py

python

```
sub =  
df[["income", "monthly_charge"]].dropna()  
stats.pearsonr(sub.income,  
sub.monthly_charge)
```

out

```
Pearson r = 0.144  
p-value   = 9.6e-04
```

Spearman: monotonic association on ranks

$$\rho = r_{\text{Pearson}}(\text{rank}(x), \text{rank}(y))$$

replace values by ranks, then correlate — **immune to outliers and curves**

spearman.py

python

```
stats.spearmanr(sub.income,  
sub.monthly_charge)
```

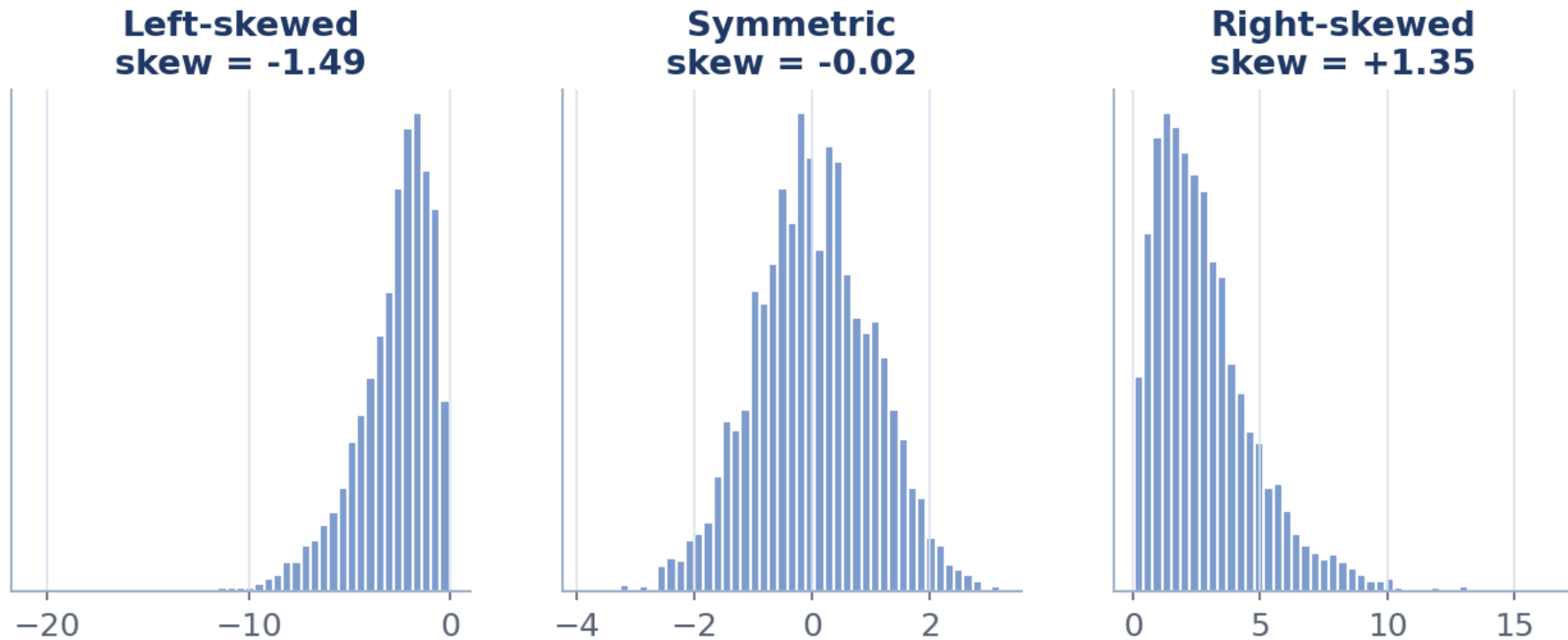
out

```
Spearman rho = 0.821  
p-value      = 2e-128
```

Same two columns!

Pearson said 0.14 (“barely related”); Spearman says 0.82. A few extreme incomes wrecked Pearson — **the rank view saw the truth.**

Anscombe's quartet — the cautionary tale



Four datasets, identical mean, variance, correlation and regression line — wildly different realities.

Correlation across the whole table

corr.py

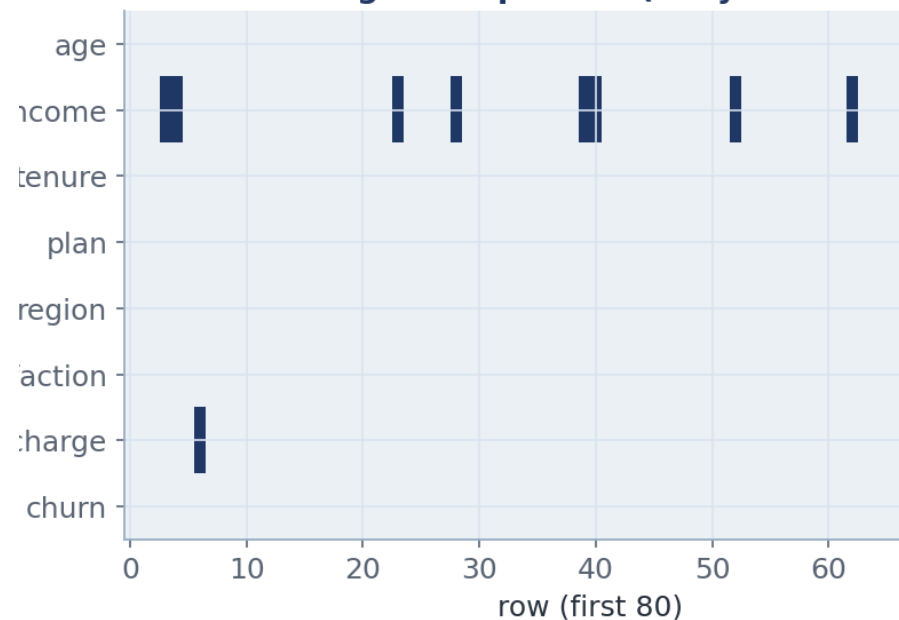
python

```
num = df.select_dtypes("number")  
num.corr() # Pearson by default
```

Correlation $\neq$ causation

A coefficient measures **co-movement**, not cause. It cannot see curves, clusters, or a lurking third variable.

Missing-value pattern (navy = missing)



Section 6

Data processing

Batch

- Fixed dataset, processed together
- Reproducible, easy to validate
- The default for training and coursework

Stream

- Data arrives continuously
- Bounded memory; must handle drift
- Statistics and models update online

The paradigm constrains which preprocessing and models are even possible — decide it before you build.

$$z = \frac{x - \mu}{\sigma} \quad (\text{z-score}) \quad ; \quad x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (\text{min-max})$$

distance- and gradient-based models need comparable scales

scale.py

python

```
from sklearn.preprocessing import  
StandardScaler  
X = df[["age", "monthly_charge"]].dropna()  
StandardScaler().fit_transform(X)  
[:3].round(2)
```

out

age	charge	age_z	charge_z
44.0	47.29	0.47	1.09
28.0	31.40	-0.78	-0.87
49.0	49.39	0.86	1.35

- **MCAR** — missing completely at random; gaps unrelated to anything. Safe-ish to drop.
- **MAR** — missingness depends on **observed** features (e.g. younger users skip income).
- **MNAR** — missingness depends on the **unobserved** value itself (high earners hide income).

Watch out

The mechanism decides whether dropping or imputing is honest. “Missing” is often **signal**, encode a was-missing flag, don’t silently erase it.

See the missingness before you fix it

missing.py

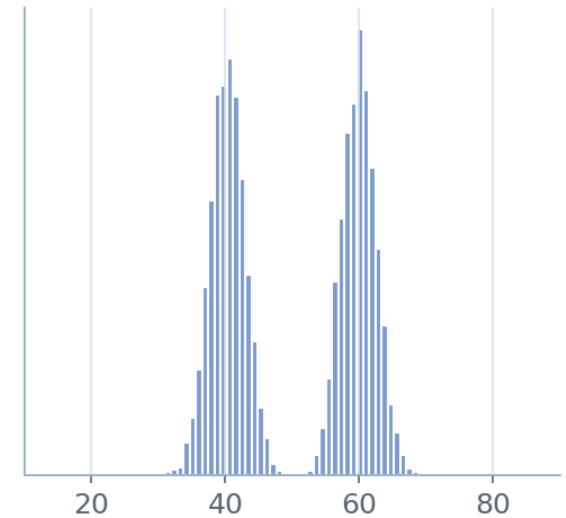
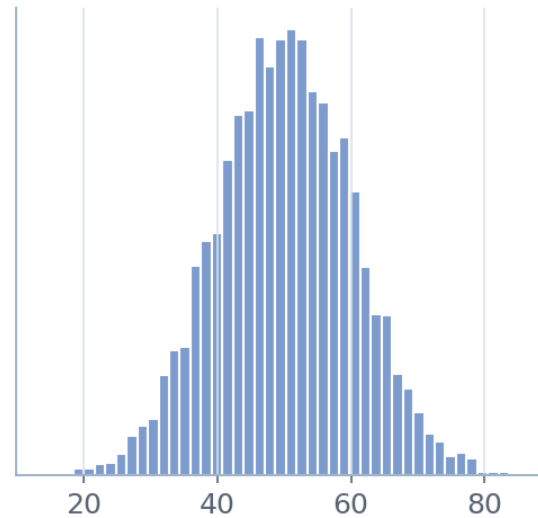
python

```
df.isna().sum()
```

out

```
age           0  
income       53  
monthly_charge 30  
plan / region 0
```

Same mean and standard deviation — completely different data
Unimodal: mean=50.2, std=10.2 **Bimodal: mean=50.2, std=10.2**



Pattern, not just count: are gaps random or clustered?

$$\hat{x}_{\text{miss}} = \text{median}(x_{\text{observed}})$$

median fill is robust; model-based fills (KNN, iterative) use the other columns

impute.py

python

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="median")
imp.fit_transform(df[["income"]])
```

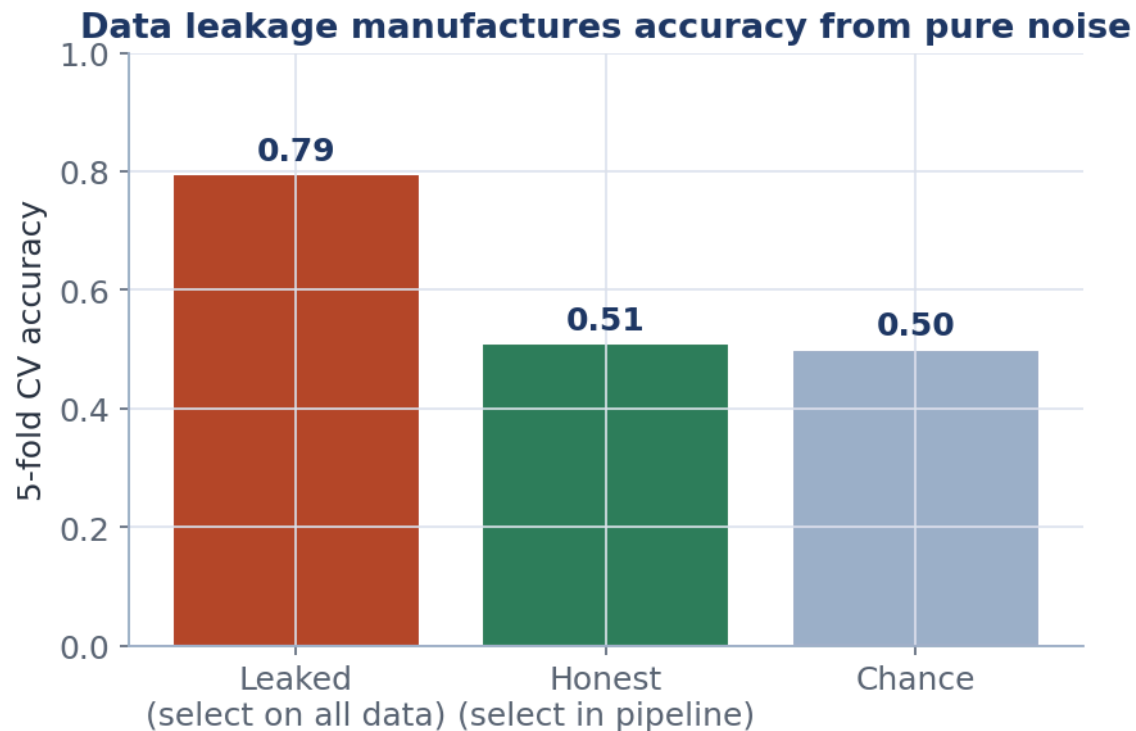
out

```
missing before = 53
median used     = 40,388
missing after   = 0
```

Watch out

Fit the imputer on **training data only** — fitting on everything leaks the test set.

Imputation changes the distribution



Median fill stacks every gap onto one value — a spike that distorts variance. KNN imputation spreads them more realistically.

$$|z_i| = \left| \frac{x_i - \bar{x}}{s} \right| > 3 \quad \text{or} \quad x < Q_1 - 1.5 \cdot \text{IQR} \quad \text{or} \quad x > Q_3 + 1.5 \cdot \text{IQR}$$

z-score assumes roughly Normal data; the IQR rule does not

outliers.py

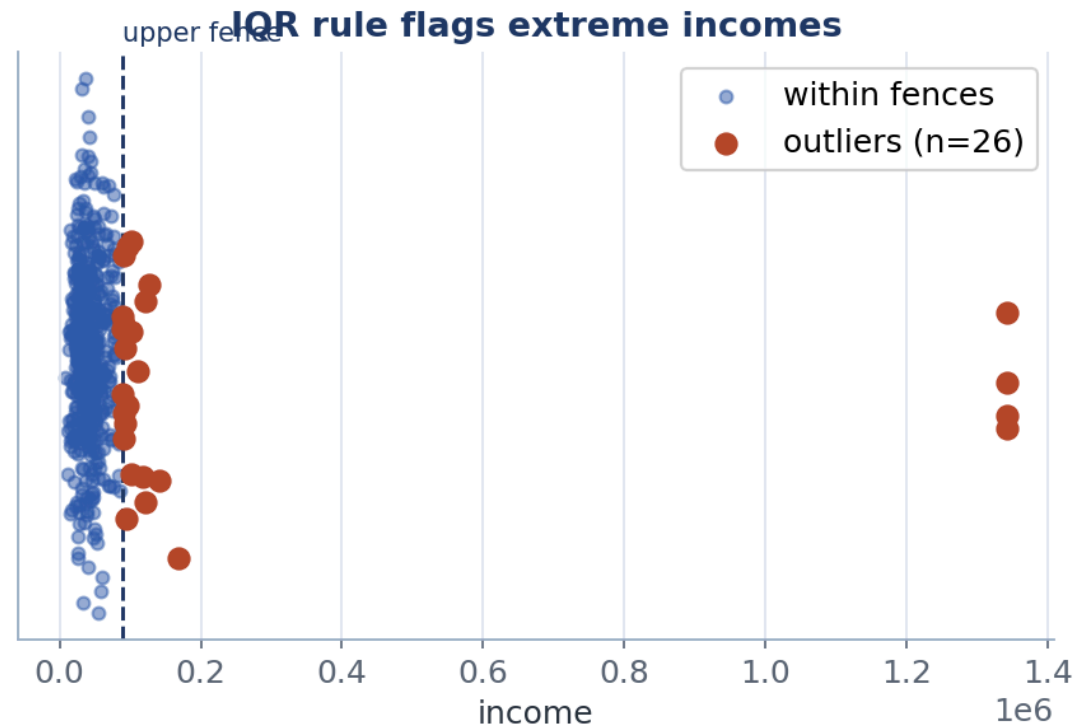
python

```
q1, q3 = inc.quantile([.25,.75]); iqr =  
q3-q1  
hi = q3 + 1.5*iqr  
inc[inc > hi]
```

out

```
upper fence = 88,154  
flagged      = 26 values  
max          = 1,342,344
```

Outliers, located



The IQR rule separates a handful of extreme incomes from the bulk — decide per cause: error, rare-but-real, or different population.

When the data is not enough

Too few examples → the model memorises noise

- Overfitting: it fits the handful of rows, not the pattern
- High variance: tiny data changes swing the model wildly
- Rare classes vanish — here churn is only **91 / 600** (15%)

Options, cheapest last

- Collect more, better data (best, but slow and costly)
- Regularise or choose a simpler model
- **Transfer learning** — reuse a pre-trained model (Weeks 9–11)
- **Data augmentation** — manufacture more training rows

The idea

Create new, **plausible** training examples by applying **label-preserving** transformations to the data you already have — enlarging and diversifying the set without collecting anything new.

A cat rotated 10° is still a cat. A customer with income nudged by €50 is the same churn label.

Augmentation depends on the data type

Data type	Label-preserving transformations
Images	Flip, rotate, crop, scale, colour jitter, add noise, cut-out
Text	Synonym swap, back-translation, random insert / delete / swap
Tabular	SMOTE (interpolate the minority class), Gaussian jitter, resample a fitted model
Audio / time-series	Time-shift, pitch-shift, add noise, time-warp
Generative (any type)	Learn the distribution, then sample: VAEs, GANs, diffusion models

The one rule that must hold

Every transformation must be **realistic** and must **not change the label**. Flipping a photo of the digit “6” turns it into a “9” — that augmentation is a bug, not a feature.

Tabular augmentation: SMOTE

$$x_{\text{new}} = x_i + \lambda(x_{z_i} - x_i), \quad \lambda \sim \text{Uniform}(0, 1)$$

step a random fraction from a minority point x_i toward a near neighbour x_{z_i} — a new synthetic row

augment.py

python

```
from imblearn.over_sampling import SMOTE
from collections import Counter
# X_tr, y_tr come from train_test_split
FIRST
print("before:", Counter(y_tr))
Xa, ya =
SMOTE(random_state=0).fit_resample(X_tr,
y_tr)
print("after :", Counter(ya))
```

out

```
before: {0: 382, 1: 68}
after  : {0: 382, 1: 382}
rows   : 450 -> 764
```

Learned augmentation: generative models

Instead of tweaking rows, **learn the data distribution and sample brand-new examples from it.**

Method	Idea
VAE	Encode to a smooth latent space, then sample and decode new examples
GAN	A generator competes with a discriminator; images, and tables via CTGAN
Diffusion	Denoise random noise into a realistic sample; best for images

Ready-made — tables: CTGAN / TVAE (SDV library) · text: paraphrase with an LLM.

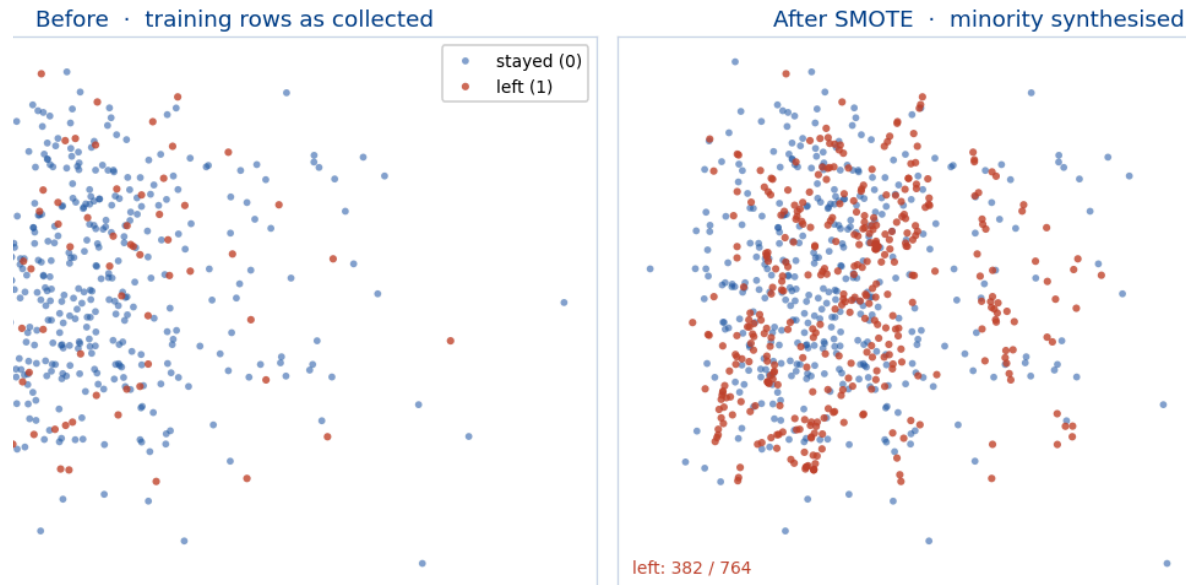
Handle with care

Generators need data to train, can **amplify existing bias**, and may invent unrealistic rows. Fit on the **training split only**.

Augment the training set only

Cardinal rule — this is leakage
Split first, then augment the **training** rows only. Never augment the validation or test set.

- Augment the test set → you **evaluate on synthetic data**
- Fit SMOTE before the split → neighbours cross it → **leakage**
- Put it **inside the Pipeline** so it only sees training folds



SMOTE fills the region between real minority points with synthetic ones — training rows only.

Section 7

Leakage-safe pipelines

The cardinal sin

Any statistic learned from the **whole** dataset before the train/test split — scaling, imputation, feature selection, encoding, augmentation — lets information from the test set seep into training. Scores look great, then collapse in production.

- **Split first**, then fit every transform on the training fold only
- Wrap the whole sequence in one Pipeline
- Cross-validate the **pipeline**, not just the final model

Leakage demo — the wrong way

200 samples, 5000 columns of **pure noise**, random labels. There is nothing to learn.

leak_wrong.py

python

```
from sklearn.feature_selection import SelectKBest, f_classif
# select the 20 "best" features using ALL the data, THEN cross-validate
Xsel = SelectKBest(f_classif, k=20).fit_transform(X, y)
cross_val_score(LogisticRegression(), Xsel, y, cv=5).mean()
```

```
$ python leak_wrong.py
```

out

```
0.795    <- 80% accuracy on noise. Impossible – and a lie.
```

Leakage demo — the honest way

leak_right.py

python

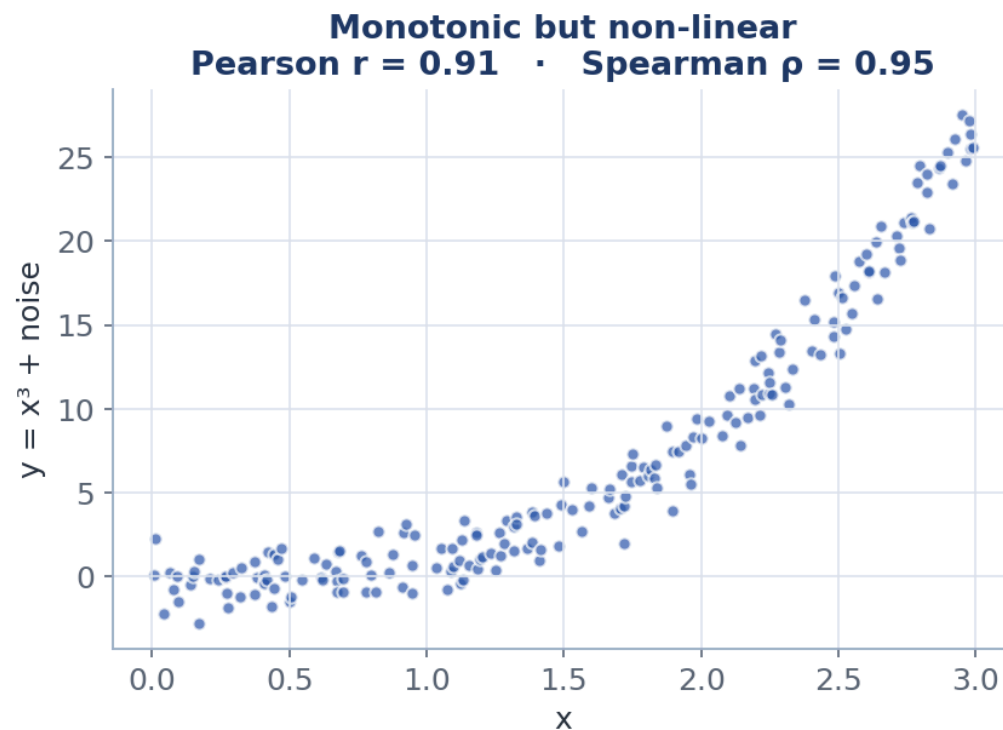
```
# selection lives INSIDE the pipeline -> refit on each train fold only
pipe = Pipeline([
    ("select", SelectKBest(f_classif, k=20)),
    ("clf", LogisticRegression(max_iter=1000)),
])
cross_val_score(pipe, X, y, cv=5).mean()
```

```
$ python leak_right.py
```

out

```
0.510 <- chance, as it must be. The pipeline told the truth.
```

The same code, two stories



One line moved — feature selection inside vs outside the pipeline — turns 0.51 into a fictional 0.79.

A leakage-safe template you can reuse

pipeline.py

python

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

num = ["age", "income", "tenure", "monthly_charge"]
cat = ["plan", "region", "satisfaction"]

prep = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())])),
    ("cat", OneHotEncoder(handle_unknown="ignore")),
])
model = Pipeline([("prep", prep), ("clf", LogisticRegression(max_iter=1000))])
# X_train/X_test came from train_test_split FIRST:
model.fit(X_train, y_train) # every transform is fit on train only
```

This single object is what you carry into Week 2 — it is already leakage-safe.

Section 8

Governance, documentation & the lab

Document the data: dictionary + datasheet

Data dictionary — every field

Field	Type
age	continuous (years)
income	continuous (€); skewed, 8.8% missing
plan	categorical
satisfaction	ordinal: Low < Med < High
churn	binary target

Datasheet — about the dataset

- **Why & how** collected; by whom, under what consent
- **Who is represented** — and who is missing
- **Licence** and appropriate use
- Known **biases and limitations**

Documentation is governance (LO5). The biases studied in Weeks 3–4 are usually baked in here, at collection time — before any model exists.

1. **Profile** your project dataset: types, distributions, missingness, outliers
2. **Build** a leakage-safe preprocessing pipeline (ColumnTransformer + Pipeline)
3. **If a class is scarce, augment the training split** (SMOTE) and re-check
4. **Document**: a data dictionary and a short datasheet

Deliverable

A runnable notebook + dictionary + datasheet, committed to your repo. Every later week builds on this.

Use the booklet as your reference and `make_assets.py` as a worked template.

- The ML workflow is a **loop**; today we own its first four stages
- **Type** dictates statistic, plot, encoding and model
- **Plot before you trust** any number — mean, SD or correlation
- Pearson sees lines, **Spearman sees order**; Anscombe says always look
- When data is scarce, **augment the training split** — never the test set
- **Split first**: leakage manufactures accuracy out of nothing
- **Documentation is governance**, and governance is where ethics begins

References & going deeper

- **Booklet** — *Data Analysis and the ML Workflow* (this week's full reference, with derivations and complete code)
- McKinney, *Python for Data Analysis* — pandas from the source
- VanderPlas, *Python Data Science Handbook* — EDA & visualisation
- Chawla et al., *SMOTE* (2002) — synthetic minority over-sampling
- Gebru et al., *Datasheets for Datasets* (2021) — the documentation standard
- scikit-learn User Guide — **Preprocessing** and **Pipelines** chapters

Run `make_assets.py` and change one thing at a time — the fastest way to build intuition.

Week 2 Supervised Learning — Regression & Classification

- From clean, documented data to **predictions**
- Regression and classification — and the **practical applications** that frame them
- Honest evaluation: the metrics, and why accuracy can mislead

Bring your Week 1 leakage-safe pipeline — Week 2 trains models on top of it.